

Reinforcement Learning with Stage-Wise Model Fusion for Sequential Object Manipulation Tasks

Yuhan Huang[†], Zijian Ning[‡], Ziyuan Wang[†], Jiaxuan Wang[†], and Xiaojun Hei^{†✉}

Abstract—Sequential robotic manipulation tasks present significant challenges, including sparse rewards and the tendency for compact models to lose previously learned skills. In this paper, we propose a multistage model fusion framework to eliminate the need for human expert demonstrations for sequential object manipulation tasks. First, simple reinforcement learning independently trains specialized meta-policies as teacher models for each sub-task. The knowledge from these teacher models is then aggregated into a lightweight Long Short-Term Memory (LSTM) student model. This fusion enables the student model to autonomously execute complex sequential tasks. This integrated method overcomes key limitations of traditional approaches, such as low training efficiency, model overfitting, and excessive reliance on expert data. We conduct evaluation experiments in simulation and on a physical robotic arm platform. Our results demonstrate superior performance and robust generalization in two challenging multistage tasks including “drawer-opening” and “coffee-making” for sequential robotic manipulation. Our proposed method may offer a scalable and efficient framework for complex sequential robot operations. The source codes are available at <https://github.com/Dian-robot/FR5-Long-Sequence-Reinforcement-Learning.git>.

I. INTRODUCTION

Reinforcement Learning (RL) and Imitation Learning (IL) have become integral to robotics, enabling robots to accomplish various tasks such as reaching, grasping, and object placement. While these methods excel at simple, single-step operations, their effectiveness diminishes for complex sequential tasks requiring multiple interdependent steps, which is a common challenge in real-world applications [1], [2].

A sequential coffee-making task, as shown in Fig. 1, may decompose into distinct sub-tasks: (1) grasping a cup, (2) placing it under a coffee machine, (3) activating the machine, (4) re-grasping the cup, and (5) delivering it. Such sequences suffer from sparse successful samples in later stages, as early failures prevent data collection for subsequent steps. Existing solutions either chain multiple models [3] or use hierarchical policies [4], but neither enables a unified model to master the full task.

In this paper, we propose a stage-wise model fusion framework to accomplish sequential object manipulation

tasks. First, we train individual teacher models for each sub-task (e.g., coffee-making’s five steps) using Proximal Policy Optimization (PPO) [5] in PyBullet. These models are then consolidated into a single student policy featuring an LSTM network [6] to capture temporal dependencies. In particular, we introduce a parallelized data sampling strategy that blends teacher demonstrations with student exploration. This hybrid approach accelerates convergence while mitigating data imbalance across task stages and preventing overfitting.

We demonstrate the generalization of the proposed method on both the coffee-making task and a drawer-opening task (Fig. 2), where a robot must (1) grasp a handle, (2) open the drawer, (3) grasp a block, and (4) place it inside.

In summary, our contributions are listed as follows:

- A model fusion framework integrating sub-task expertise into a unified policy for sequential manipulation.
- A data sampling scheme balancing teacher guidance and autonomous exploration, alleviating stage-wise data imbalance and enhancing planning efficiency.
- Simulation and real-world validation with open-source codes.

II. RELATED WORK

The question of how to enable reinforcement learning (RL) to handle temporally extended tasks is a hot research topic, and hierarchical reinforcement learning (HRL) schemes are an early and promising approach. The original Option-Critic Architecture [7] introduced the concept of hierarchy, where a high-level policy determines the next skill and a low-level policy selects parameters for that skill. Many subsequent works built upon this foundation, for example, FeUdal Networks [8] refined intrinsic reward mechanisms for better exploration, Nachum et al. [9] explored how to tackle problems without explicitly setting intermediate goals, and Levy et al. [3] proposed more stable convergence methods. Although these approaches have clear and concise mathematical logic, they mostly focus on sparse reward environments and can only perform simple game tasks.

Another highly effective method is HER, which provides the agent with more useful learning information by modifying the goals in the replay buffer. Some improvements, such as Relay Hindsight Experience Replay [10], have extended HER to sequential tasks through a self-guided policy using a meta-action model. As imitation learning gained popularity, some methods, such as Relay Policy Learning [11], introduced human demonstrations as additional supervision to goal-conditioned reinforcement learning, allowing robots to complete tasks in more complex scenarios. However,

[†]School of Electrical, Information and Communication Engineering, Huazhong University of Science and Technology, Wuhan, China.

[‡]School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China.

E-mail: {woshihg, aurora.nzj, wangzyprince, wangjx_hust, heixj}@hust.edu.cn

This work was supported in part by the Huazhong University of Science and Technology Teaching Research Fund (No. 2025YBO04) and the College Student Innovation Project under Grant No. S202510487156. Xiaojun Hei is the corresponding author.

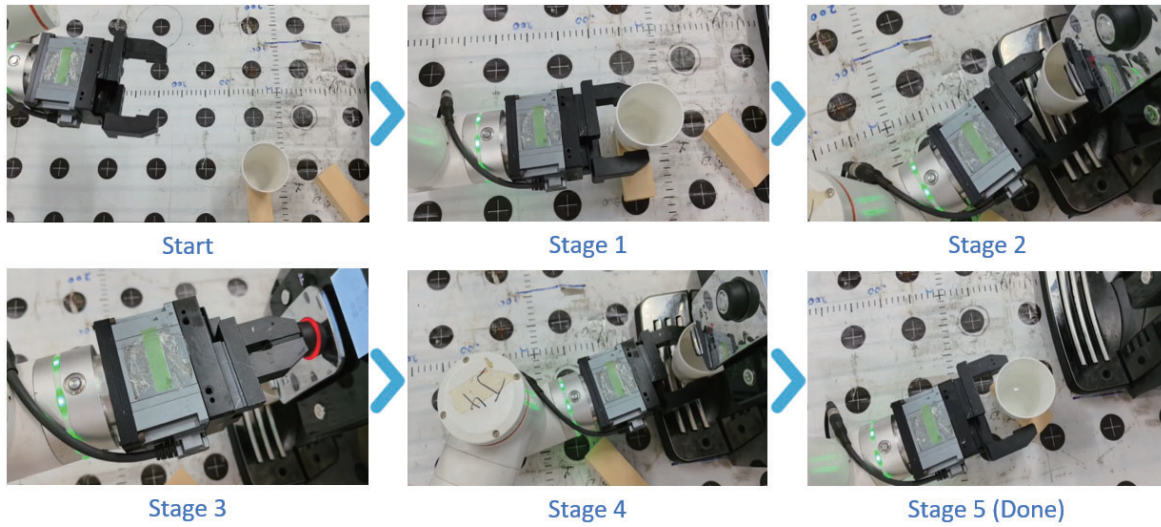


Fig. 1. A sequential coffee-making task in the real world

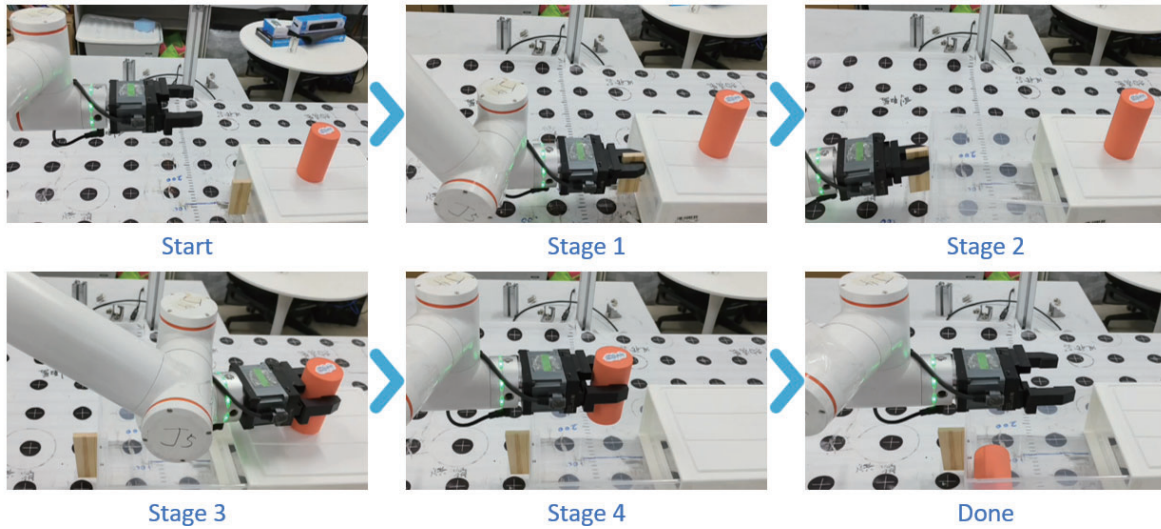


Fig. 2. A sequential drawer-opening task in the real world

these methods require large amounts of expert data and the collection of such data is a significant challenge. In contrast, our work focuses on sequential tasks in real-world robotic arm applications. It achieves objectives that are unattainable by self-guided policies alone, without the need for additional expert demonstration data.

III. METHOD

Our task is designed for a real-world scenario, where we build simulation models of a real robot and manipulated objects within the PyBullet [12] simulation platform, as shown in Fig .3.

Data collection and policy training are performed in the simulation environment, followed by validation in the real robot. Each stage of the task can be abstracted as commanding the robotic arm's end-effector to reach a specific position. Since the gripper's open/close state is binary, we directly control it during experiments and do not include it in our discussion.

Our proposed method consists of two main steps. In the first step, we use reinforcement learning to acquire multiple meta-policies, each of which is capable of completing a specific stage of a sequential task. In the second step, we employ a data fusion sampling method that leverages these meta-policies to teach a student model to independently complete the entire sequential task. This second step represents our main innovative contribution.

A. Learning Meta-Policies

We adjust the initial positions of the robotic arm and the states of the manipulated objects according to the specific requirements of the task. We use the PPO algorithm from the stable-baselines3 library [13] for training. PPO is a widely used reinforcement learning method that ensures stable policy convergence by constraining the magnitude of policy updates. For our policy network, we employ a two-layer Multilayer Perceptron (MLP) with 64 neurons in each layer. Based on the key reinforcement learning settings [2],

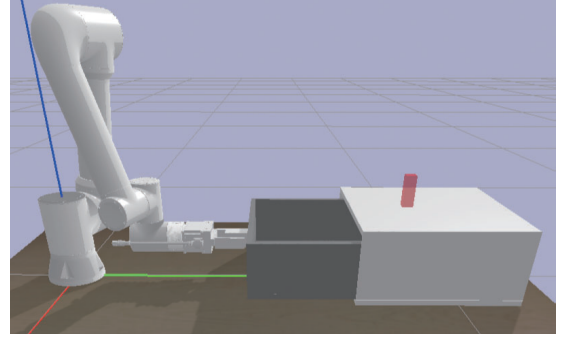
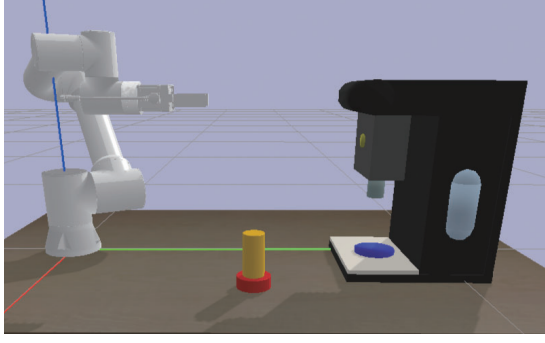


Fig. 3. Simulation environments for two sequential object manipulation tasks. (a) Coffee-making Task. (b) Drawer-opening Task.

we have optimized them for our specific task. Our final settings are defined as follows:

1) *Observation*: The observation space is defined by the joint angles of the robotic arm, the pose (position and orientation of the Euler angle θ) of the end-effector, and the position of the target point. This is expressed as follows.

$$Joint = [J_1, J_2, \dots, J_6] \quad (1)$$

$$Effector_Pose = [x_{end}, y_{end}, z_{end}, \theta_x, \theta_y, \theta_z] \quad (2)$$

$$Target = [x_{target}, y_{target}, z_{target}] \quad (3)$$

$$Observation = [Joint, Effector_Pose, Target] \quad (4)$$

2) *Actions*: The action space is defined as the change in the joint angles of the robotic arm in the current time step. After an action is executed, the robotic arm moves to a new position corresponding to the current joint angles plus this differential value.

$$Action = [\Delta J_1, \Delta J_2, \dots, \Delta J_6] \quad (5)$$

3) *Reward*: The reward function includes a dense distance reward $r_{distance}$, a sparse success reward $r_{success}$, and an additional pose penalty r_{pose} .

$$R_{total} = r_{distance} + r_{success} + r_{pose} \quad (6)$$

The reward $r_{distance}$ is defined as the Euclidean distance d between the gripper's center position and the target point's position. Here, d_{last} denotes the value of this distance before the action was executed.

$$r_{distance} = \begin{cases} 0, & \text{if step_num} = 0 \\ 5000 \cdot (d_{last} - d), & \text{elif } d \leq 0.1 \\ 1000 \cdot (d_{last} - d), & \text{else} \end{cases} \quad (7)$$

The reward $r_{success}$ is a sparse reward given at the conclusion of the task. It is a positive value for successful completion and a negative value for task failure or undesirable actions, where task failure specifically includes collision with the environment.

$$r_{success} = \begin{cases} 100, & \text{succeeded} \\ -10, & \text{failed} \end{cases} \quad (8)$$

The pose reward (r_{pose}) measures the end-effector's Euler angle deviation from a desired orientation. This desired orientation is represented by a standard Euler angle vector, θ_0 ,

which corresponds to the gripper facing horizontally forward. The reward is calculated as the negative squared Euclidean distance between the current orientation $\theta = (\theta_x, \theta_y, \theta_z)$ and the standard orientation θ_0 :

$$r_{pose} = -0.01 \|\theta - \theta_0\|^2 \quad (9)$$

Although designing these single-stage tasks may seem cumbersome, the process is actually straightforward. By appropriately arranging the objects in the simulation, different task stages can share the same environment configuration and reward logic. The only effort required is to establish the initial position of the robotic arm, a process that can even be easily automated by generative artificial intelligence.

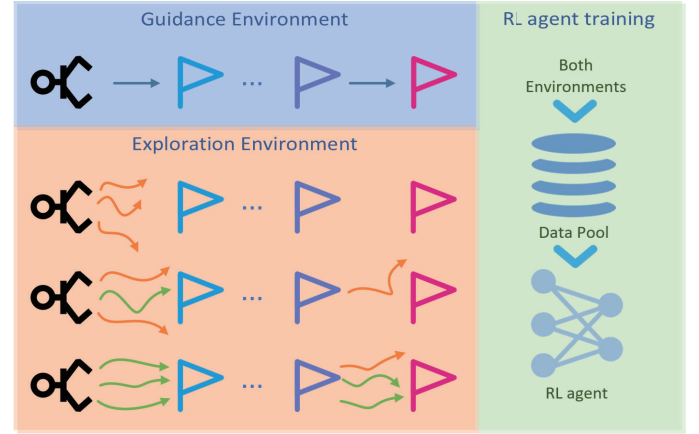


Fig. 4. **Training mode in sequential task**: The training employs parallel environments. In the guidance environment, the model consistently completes the task successfully. In the exploration environment, a visual key of yellow for failure and green for success is used to illustrate the possible states at different training stages.

B. Reinforcement Learning with Stage-Wise Model Fusion

After acquiring a series of single-task models, we designate them as teacher model within a teacher-student framework. The model we train to complete the entire sequential task serves as a student model.

The student model is also trained using the PPO algorithm. A key architectural difference is the use of an LSTM network, which is better suited for sequential tasks, instead of a standard MLP. Unlike a simple MLP that struggles with complex and dynamic inputs, the LSTM introduces temporal

context. This allows it to make action decisions based on the entire action sequence, rather than relying solely on a single-frame state. The model is configured as a two-layer deep LSTM network, with each layer containing 256 hidden units. Although this results in a slight increase in the number of parameters, the resource requirements remain very low.

In each training step, we simultaneously obtain the outputs of both the teacher model and the student model from the same environmental observation, denoted respectively as $\mathbf{A}_t$ and $\mathbf{A}_s$.

1) *Observation*: The observation space for our student model is augmented to handle the challenge posed by sequential tasks, where similar observations can demand different actions depending on the task stage. Based on the previously defined Eq.(4), we added an additional observation dimension *Stage* to represent the robot's current stage. n is a discrete integer representing policy is in the n^{th} stage. N represents the total stages of a complete task, e.g., $stage_max = 5$ in the coffee-making task.

$$Stage = \frac{n}{N} \quad (10)$$

$$Observation = [Joint, Effector_Pose, Target, Stage] \quad (11)$$

2) *Actions*: The action space of the student model is the same as that of the teacher model, as defined in Eq.(5).

3) *Reward*: The environmental information is implicitly encoded within the parameters of the teacher model. Therefore, the primary objective of this stage is to achieve alignment between the output of the student and teacher model. The reward function for this phase is highly simplified and is defined by directly penalizing the Euclidean distance between the two models' outputs, with the reward r_{stu} given by:

$$r_{stu} = -\frac{\|\mathbf{A}_s - \mathbf{A}_t\|_2}{10} \quad (12)$$

In our multi-stage simulation environment, the gripper can only proceed to the next phase after successfully completing the current one. We designed two different environments to sample data in parallel, as shown in Fig. 4. One is the Guided Environment, where the robotic arm executes actions precisely according to the teacher model output. The other is the Exploration Environment, where the robotic arm performs actions based on the student model's output. Both environments adhere to the reward scheme described above. A key feature is that in the Guided Environment, although the robot follows the teacher's instructions, the student model's own output is still recorded and used for the reward calculation.

The two environments work together to great effect. The task is completed by setting a specific ratio γ_E^G between Guidance and Exploration in environments. It can be flexibly chosen based on empirical results and we found that the optimal ratio γ_E^G for coffee-making task is 0.25, and for drawer-pulling task, it is 0.5. The Guided Environment allows us to sample full-trajectory data, which prevents the student model from wasting too much time on early-stage tasks and becoming overfitted, thereby failing to progress to

later stages. At the same time, the Exploration Environment ensures that the student model samples data from a wider range of state space. This is crucial for supplementing the teacher's dataset with failure cases and other unforeseen situations, which in turn significantly improves the model's robustness. The complete algorithm is presented as Algorithm 1.

Algorithm 1 Long Sequence Model Training

Require: Teacher model $M_1, \dots, M_N$ where N is the total number of stages; Student model M_s with parameters θ_{M_s}

- 1: **Initialize:** Set up parallel environments
- 2: **for** $epoch \leftarrow 1$ to max_epochs **do**
- 3: $n \leftarrow 1, done \leftarrow False$
- 4: **while** not done **do**
- 5: Get $\mathbf{A}_s$ from M_s
- 6: Get $\mathbf{A}_n$ from M_n
- 7: Compute Reward with Eq.(12)
- 8: **if** In Exploration Environment **then**
- 9: Take Action $\leftarrow \mathbf{A}_s$
- 10: **else if** In Guidance Environment **then**
- 11: Take Action $\leftarrow \mathbf{A}_n$
- 12: **end if**
- 13: Update θ_{M_s}
- 14: **if** stage n is successful and $n < N$ **then**
- 15: $n \leftarrow n + 1$
- 16: **else**
- 17: $done \leftarrow True$
- 18: **end if**
- 19: **end while**
- 20: **end for**

IV. PERFORMANCE EVALUATION

In this section, we conduct simulation and testbed experiments to evaluate the performance of our method.

A. Experiment Setup

Our robotic arm platform utilizes a high-precision, six-axis FAIRINO FR5 manipulator equipped with a DH Robotics PGI-140-80 two-finger parallel gripper. A coordinate system is established with its origin at the center of the robot's base, as depicted in Fig. 5.

The coffee and drawer tasks are configured as previously described. Note that the position of the manipulated objects is provided by a computer vision system; however, this system is external to the policy framework and will not be discussed further in this paper. The success condition for each experimental stage is defined as the robotic arm completing its trajectory without colliding with any environmental components, and its final position being within a 1 cm tolerance of the target. Upon successful completion of a stage, the system proceeds to the next; otherwise, the stage is reinitialized.

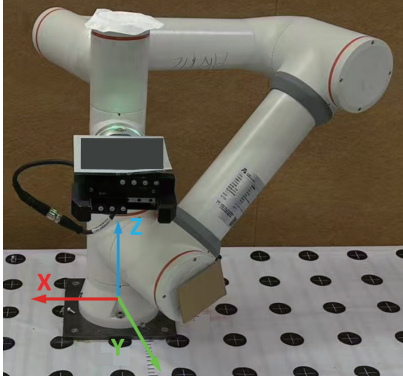


Fig. 5. FAIRINO FR5 with DH Robotics PGI-140-80

This task presents significant challenges. During execution, the gripper must interact with objects within the simulated environment. For example, when grasping a cup, the robot must avoid knocking it over. When pulling a drawer, it must maintain a parallel retraction motion.

TABLE I. Task Position Targets

Coffee	Target Position		
1: Initial Grasp	$x_1=[0.2,0.3]$	$y_1=[0.5,0.6]$	$z_1=0.08$
2: Machine Placement	$x_2=[0.1,0.2]$	$y_2=[0.8,0.9]$	$z_2=0.09$
3: Button Press	$x_3=x_2$	$y_3=y_2-0.55$	$z_3=0.3$
4: Re-grasp Cup	$x_4=x_2$	$y_4=y_2$	$z_4=z_2$
5: Final Placement	$x_5=[0.25,0.35]$	$y_5=[0.6,0.7]$	$z_5=0.09$
Drawer	Target Position		
1: Grasp Handle	$x_1=[-0.05,0.05]$	$y_1=[0.7,0.77]$	$z_1=0.1$
2: Pull Drawer Open	$x_2=x_1$	$y_2=y_1-0.27$	$z_2=0.1$
3: Grasp Target	$x_3=[-0.1,0.1]$	$y_3=[0.62,0.74]$	$z_3=0.23$
4: Place Target	$x_4=x_2$	$y_4=y_2+0.15$	$z_4=z_3$

Furthermore, the target positions for these tasks are not fixed. They are sampled from a uniform distribution within a specified range, as detailed in Table I.

The process of obtaining the meta-policies has been extensively studied and is a straightforward problem. Therefore, it is not the focus of our research and will not be discussed in detail. The meta-policies we obtained through our method's setup all achieved a success rate of 1.0.

B. Experiment Results

1) *Training Details:* We conducted our training using an NVIDIA GeForce RTX 3070 GPU and an Intel i7-12700K CPU. The simulation environment was configured with the following parameters: a learning rate of 0.0003, a batch size of 64, a discount factor of 0.99, a PPO clipping parameter set to 0.2. An update of 10 epochs was performed every 2048 steps. Each experimental episode consisted of 30,000 steps.

We conducted experiments to address the challenge of sequential tasks. The performance metric used was the success rate of the student policy in completing the tasks. Each data

point was obtained from 50 test runs, and the target position was re-sampled for each test. In addition to evaluating the success rate for the full task, we performed separate tests for each stage to investigate and demonstrate why conventional methods are unsuitable for sequential tasks. To improve efficiency, 16 environments were run in parallel.

2) *Simulation Results:* To demonstrate the effectiveness of our method for sequential tasks, we use different ratios θ between Guided and Exploration in environments. All methods use the same reward function and observations.

- **Exploration-Only Method:** The γ_E^G is set to 0, which means that it allows the student model to execute the output action completely autonomously, without interference from the teacher model.
- **Guidance-Only Method:** The γ_E^G is set to 1, which means that only the decisions of the teacher model were adopted. The student learned from the correct data sampled by the teacher model.
- **Our Proposed Method:** The γ_E^G is set to 0.25 in the Coffee-making Task and 0.5 in the Drawer-opening Task. It combines the teacher model's decisions with the student model's own exploration.

The experimental results are shown in Fig. 6. When the Exploration-Only method was used for reinforcement learning training, the student policy quickly completed the early-stage tasks. As seen in stages 1 and 2 of the coffee task and stage 1 of the drawer task, this method was the first to achieve a success rate of 1.0 among all methods. However, because the policy focused on sampling data from the early stages, it struggled to fit to new data in subsequent stages. The coffee task remained stuck at stage 3, and the drawer task at stage 2.

When the Guidance-Only method was used for reinforcement learning training, the student model simultaneously learned all stages of the task. This method achieved a non-zero success rate in every stage. However, due to the lack of exploratory data in the samples, the model struggled to correct for minor deviations. Aside from stage 4 of the coffee task and stage 2 of the drawer task, the overall success rate was very low, and the method was unable to successfully complete the full task.

In contrast, our method demonstrated a clear advantage. As shown in Fig. 6, our approach progressively learns tasks stage by stage, starting from stage 1 in both tasks, without forgetting previously acquired knowledge. After approximately 100 episodes, it gradually learned to complete the full tasks, achieving success rates of 1.0 and 0.96 for the Coffee-making and Drawer-opening tasks, respectively. This performance fully validates the effectiveness of our proposed method.

3) *Testbed Results:* We addressed real-world problems from the outset and ultimately reproduced the tasks on a physical robotic arm. For safety reasons during the button pressing step, we controlled the gripper to move directly forward to press the button after reaching the position in front of it. Since different types of coffee have different preparation times, the time interval for waiting for the coffee

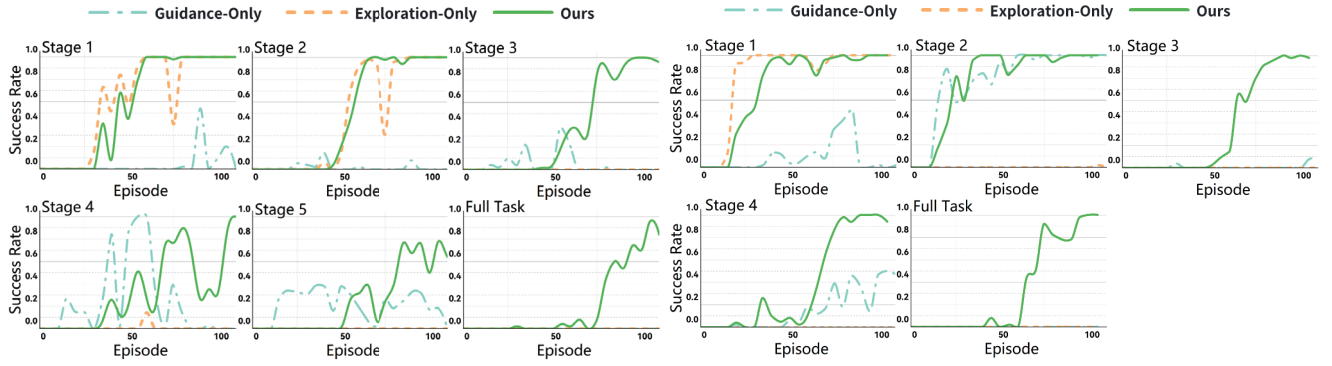


Fig. 6. Simulation Results. (a) Coffee-making. (b) Drawer-opening.

to be prepared is set manually. All other settings, including random sampling of the position of the target object, were identical to the simulation.

TABLE II. Testbed Results

Task	Number of Experiments	Success Rate
Coffee-making	20	0.85 (17 / 20)
Drawer-opening	20	0.90 (18 / 20)

Using the model with the highest success rate, we conducted 20 experiments in the real environment, applying the same success criteria as in the simulation. Despite the fact that tasks like grasping a cup and pulling a drawer involve interactions with the real world that are difficult to fully replicate in a simulation, the robotic arm's performance in the real world was consistent with its performance in the simulated environment. Although the cup was more easily knocked over in the real world, our experiments still showed a high success rate, achieving 0.85 for the Coffee-making Task and 0.90 for the Drawer-opening Task. This shows that our experimental design has practical application potential.

V. CONCLUSION

Based on a sim-to-real workflow, in this paper we propose a novel data sampling method within a reinforcement learning framework. By setting different data sampling strategies in parallel simulation environments, we leverage an expert model to quickly collect successful trajectories and use a student model for free exploration to maintain corrective capabilities. We employed a reinforcement learning model with a small number of parameters to successfully create an agent capable of completing sequential tasks. This fusion of data from different sampling strategies provides inspiration for related research with the following remarks: 1) LSTM models are advantageous for reinforcement learning tasks,

and it is feasible to use models with a small number of parameters to accomplish sequential tasks. 2) The data sampling process in reinforcement learning has a direct and profound impact on the final result, indicating that optimizing this stage holds significant potential.

ACKNOWLEDGMENT

We would like to thank Yefan Lin and Lujia Zhang for their contributions in the initial stage of this work.

REFERENCES

- [1] A. Lobbezoo et al., "Reinforcement learning for pick and place operations in robotics: A survey," *Robotics*, vol. 10, no. 3, Sep 2021.
- [2] Z. Wang et al., "Deep reinforcement learning for sim-to-real transfer in a humanoid robot barista," in *IEEE International Conference on Robotics and Biomimetics*, Dec 2024, pp. 931–936.
- [3] M. Vavrecka et al., "Modular reinforcement learning in long-horizon manipulation tasks," in *Artificial Neural Networks and Machine Learning (ICANN)*, 2024, pp. 299–312.
- [4] Z. Li et al., "Learning a skill-sequence-dependent policy for long-horizon manipulation tasks," in *IEEE International Conference on Automation Science and Engineering*, Aug 2021, pp. 1229–1234.
- [5] J. Schulman et al., "Proximal policy optimization algorithms," Aug 2017, arXiv.
- [6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [7] P.-L. Bacon, J. Harb, and D. Precup, "The option-critic architecture," *AAAI Conference on Artificial Intelligence*, vol. 31, no. 1, Feb 2017.
- [8] A. S. Vezhnevets et al., "FeUdal networks for hierarchical reinforcement learning," in *International Conference on Machine Learning*, Jul 2017, pp. 3540–3549.
- [9] O. Nachum et al., "Data-efficient hierarchical reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [10] Y. Luo et al., "Relay hindsight experience replay: Self-guided continual reinforcement learning for sequential object manipulation tasks with sparse rewards," *Neurocomputing*, vol. 557, p. 126620, Nov 2023.
- [11] A. Gupta et al., "Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning," in *Proceedings of the Conference on Robot Learning*, May 2020, pp. 1025–1037.
- [12] E. Coumans and Y. Bai, "Pybullet: a python module for physics simulation for games, robotics and machine learning," 2016–2021.
- [13] A. Raffin et al., "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.